

Improving Symbolic Regression with Interval Inversion and Constant Stabbing

Maarten Keijzer

July 2026

Abstract

1 Introduction

2 Semantic inversion using intervals

Semantic backpropagating crossovers [?] use analytical inversion to transform root level targets into surrogate targets that are used in a subsequent search for a good sub-tree for that position. While this preserves the location of the target value, it does not preserve the error structure in general; a subtree that is a better fit to the surrogate target according to some metric need not lead to a better overall error at the root level. Only when the new sub-tree dominates the original sub-tree on all surrogate targets can a guaranteed improvement be realized.

Here, instead of using a squared error or related measure for judging different trees, we use interval bands. Target values are bound with one or more interval bands, with each band representing a tolerance for being off-target.

Given a set of targets $[t_1, \dots, t_n]$, the symbolic regression problem is reformulated to hitting targets $[(t_1 - \epsilon, t_1 + \epsilon), \dots, (t_n - \epsilon, t_n + \epsilon)]$, or in general hitting a set of intervals $[(l_1, u_1), \dots, (u_n, l_n)]$

When using a single band, the metric reduces to Koza’s original hits metric [?].

With multiple equally spaced bands $(\epsilon, 2\epsilon, \dots)$, the fitness metric is roughly equivalent to ϵ -lexicase selection [?] with pre-set bands.

The advantage of using interval inversion is that hits are maintained. If a subtree hits (stabs) a surrogate target’s interval, it is guaranteed that the overall tree will also hit the original target, and therefore scores a hit when inserted. This guarantees that the metric is invariant under inversion.

However, using these surrogate targets for selecting sub-trees is not immediately useful. The efficiency gains when doing sub-tree selection at depth d in

the tree will only save dn evaluations compared to a naive insertion of these same sub-trees in the overall tree directly, and evaluate it at the root level.

3 Computing interval inversions

Given a node whose output must lie in a target interval $[l, u]$, interval inversion maps that requirement backward through the node's operation to produce a surrogate target for the child subtree. Each of the n data points is handled independently, so the result is a vector of surrogate intervals, one per point.

3.1 Binary operators: multiplication

For a node computing $z = x \cdot y$, where the right subtree evaluates to scalar y_j at data point j and the target is $[l_j, u_j]$, the surrogate target for the left subtree is the set of values x satisfying $x \cdot y_j \in [l_j, u_j]$.

When $y_j > 0$, scaling by $1/y_j$ preserves the interval order:

$$x \in \left[\frac{l_j}{y_j}, \frac{u_j}{y_j} \right].$$

When $y_j < 0$, multiplication by a negative number reverses the order, so the bounds swap:

$$x \in \left[\frac{u_j}{y_j}, \frac{l_j}{y_j} \right].$$

When $y_j = 0$, the product is zero regardless of x ; the data point carries no information about the left child and contributes an invalid sentinel that is excluded from the stabbing sweep.

As a concrete example, suppose the target at data point j is $[0.5, 1.5]$. If the right subtree evaluates to $y_j = 2$, the surrogate target for the left child is $[0.25, 0.75]$. If instead $y_j = -2$, the interval is reflected and the surrogate target becomes $[-0.75, -0.25]$. The surrogate target for the right subtree is derived symmetrically.

3.2 Unary operators: sine

Monotone unary functions produce a single surrogate interval per target interval. Periodic functions are different: because $\sin$ has two monotone branches in each period, inversion produces *two* disjoint surrogate intervals per target interval.

Over one period, $\sin$ is increasing on $[-\pi/2, \pi/2]$ and decreasing on $[\pi/2, 3\pi/2]$. For a target $[l, u] \subseteq [-1, 1]$ the two pre-image branches are:

$$\text{Branch 1 (increasing): } x \in [\arcsin(l), \arcsin(u)], \quad (1)$$

$$\text{Branch 2 (decreasing): } x \in [\pi - \arcsin(u), \pi - \arcsin(l)]. \quad (2)$$

Targets that fall entirely outside $[-1, 1]$ are unreachable; an empty set is returned and the data point is excluded from the stabbing sweep.

As a concrete example, consider target $[0.5, 0.8]$. With $\arcsin(0.5) \approx 0.524$ and $\arcsin(0.8) \approx 0.927$, the two surrogate intervals are:

Branch 1: $[0.524, 0.927]$,

Branch 2: $[\pi - 0.927, \pi - 0.524] \approx [2.214, 2.618]$.

Both are genuine pre-images: $\sin(0.7) \approx 0.644$ (Branch 1) and $\sin(2.4) \approx 0.676$ (Branch 2) both lie within $[0.5, 0.8]$.

The two branches represent qualitatively different families of solution. A subtree whose output hovers near 0.7 and one whose output hovers near 2.4 are equally valid, and an additive constant that aligns the first may differ substantially from one that aligns the second. Each data point therefore contributes two surrogate intervals to the maximum-overlap stabbing problem rather than one. The algorithm is unchanged; it processes more intervals, and the wider candidate set increases the chance of finding a stabbing constant that covers many points simultaneously.

These two examples cover the mechanics behind all function inversions used here: addition, subtraction and division get similar treatment as multiplication; monotonic functions such as log, exp and sqrt have direct inverses (with bounds check), and functions such as abs and square follow the multi-branch of the sine function. Repeated application of these functions thus scatter the intervals and the number of intervals considered for a single point is capped at 16 intervals.

4 Maximum overlapping intervals

Stabbing problems are studied in computational geometry to find the number of areas a ray, line, or general shape intersects with [?].

In symbolic regression with interval (surrogate) targets, the areas of interest are the (surrogate) target intervals, and trees try to hit (stab) as many intervals as possible. When all intervals are stabbed, a 100% hit rate is achieved.

Finding an interval that intersects with the maximum number of intervals from a set is an $O(n \log n)$ operation in the number of intervals considered; i.e.; given a set of interval targets, the algorithm finds a range of constants, each constant from that range stabbing the maximum number of intervals that are achievable.

Therefore, given a subtree T producing evaluations x against a surrogate target $[(l_1, u_1), \dots, (l_n, u_n)]$, the maximum overlapping interval algorithm gives us a range of values $c \in (l, u)$ for which $x + c$ stabs most intervals, i.e., maximizes hits.

Given the invariance property of interval inversion on the number of hits, this makes it possible to find optimal additive constants anywhere in the tree. Instead of using unguided or gradient search to iteratively find a good constant, the optimal constant(s) can be found in $O(n \log n)$ time. When this routine is used for crossover or mutation, it will ensure that a new subtree will be brought into the right range, stabbing at least one interval. Fully destructive crossovers are no longer possible.

Finding also the scale instead of just the intercept has complexity $O(n^2)$, and is therefore too expensive to run in the inner loop of a symbolic regression algorithm.

4.1 Selecting stabbing constants

The maximum interval overlap routine returns one or more intervals. Any value within that range is a valid constant for maximizing the number of hits. When selecting a constant, it is first checked if the value 0 is part of the range. If that is the case, the value is chosen for the constant because it reduces structural complexity ($x + 0 = x$).

Otherwise, another check is performed to see if there are integers that are optimal. If there are such integers, an integer is drawn uniformly from the intervals. This implements a preferential bias towards integer constants.

Finally, if there is no zero, nor integers in the range, a constant is drawn uniformly from the allowable ranges.

5 Stabbing crossover and mutations

Using the machinery of interval inversion and constant stabbing, corresponding crossover and mutation operators are defined using an **aligned-insert** routine. It inserts the tree at a position, adds an optimal constant to put it in range, and repeats this on the path upward, drawing fresh constants along the way. Redrawing constants repositions the surrogate targets for other branches, possibly making the overall problem easier to solve.

5.1 The recursive-stab algorithm

Every node carries an additive constant c , so a node with function f evaluates as $f(\cdot) + c$. The algorithm recurses from the root of R toward position i , propagating surrogate targets downward and updated evaluations upward, re-optimising each ancestor’s additive constant on the way back.

MAXOVERLAP runs the $O(n \log n)$ interval-stabbing sweep and returns the optimal constant c^* (selected per Section 4.1). The total cost is $O(d \cdot n \log n)$ where d is the insertion depth, dominated by d calls to MAXOVERLAP. Nodes off the path are never re-evaluated.

6 Methods and materials

The previous sections laid out the main contribution of the paper. This section provides further algorithmic details and describes the experimental setup.

Structural complexity The structural complexity of a tree is computed as the number of nodes in a tree. As every node has an accompanying constant,

Algorithm 1 $\text{ALIGNEDINSERT}(N, D, i, \mathbf{x}, \mathcal{T})$

Require: recipient node N , donation subtree D , index i , inputs $\mathbf{x}$, interval targets $\mathcal{T} = [(l_j, u_j)]_{j=1}^n$

Ensure: updated node N' , evaluations $\mathbf{v} \in \mathbb{R}^n$

```

1: if  $i = 1$  then                                      $\triangleright$  insertion point reached
2:    $\mathbf{v} \leftarrow \text{eval}(D, \mathbf{x}) - D.c$ 
3:    $c^*, \mathbf{v} \leftarrow \text{MAXOVERLAP}(\mathcal{T} - \mathbf{v})$ 
4:   return  $D.\text{withConstant}(c^*), \mathbf{v} + c^*$ 
5: end if
6: if  $N$  is a UnaryNode with function  $g$  then
7:    $\mathcal{T}' \leftarrow g^{-1}(\mathcal{T} - N.c)$                       $\triangleright$  interval inverse, pointwise
8:    $N'_{\text{child}}, \mathbf{v}_{\text{child}} \leftarrow \text{ALIGNEDINSERT}(N.\text{child}, D, i - 1, \mathbf{x}, \mathcal{T}')$ 
9:    $\mathbf{v} \leftarrow g(\mathbf{v}_{\text{child}})$ 
10: else if  $N$  is a BinaryNode with function  $\oplus$  and  $i \leq 1 + |N.\text{left}|$  then
11:    $\mathbf{v}_R \leftarrow \text{eval}(N.\text{right}, \mathbf{x})$ 
12:    $\mathcal{T}' \leftarrow \text{leftInverse}(\oplus, \mathcal{T} - N.c, \mathbf{v}_R)$ 
13:    $N'_{\text{left}}, \mathbf{v}_L \leftarrow \text{ALIGNEDINSERT}(N.\text{left}, D, i - 1, \mathbf{x}, \mathcal{T}')$ 
14:    $\mathbf{v} \leftarrow \mathbf{v}_L \oplus \mathbf{v}_R$ 
15: else                                                  $\triangleright$  insertion is in right subtree
16:    $\mathbf{v}_L \leftarrow \text{eval}(N.\text{left}, \mathbf{x})$ 
17:    $\mathcal{T}' \leftarrow \text{rightInverse}(\oplus, \mathcal{T} - N.c, \mathbf{v}_L)$ 
18:    $N'_{\text{right}}, \mathbf{v}_R \leftarrow \text{ALIGNEDINSERT}(N.\text{right}, D, i - 1 - |N.\text{left}|, \mathbf{x}, \mathcal{T}')$ 
19:    $\mathbf{v} \leftarrow \mathbf{v}_L \oplus \mathbf{v}_R$ 
20: end if
21:  $c^*, \mathbf{v} \leftarrow \text{MAXOVERLAP}(\mathcal{T} - \mathbf{v})$               $\triangleright$  re-optimize this node's constant
22: return  $N'.$  $\text{withConstant}(c^*), \mathbf{v} + c^*$ 

```

an extra count for a non-zero constant is added to the structural complexity of a tree.

Complexity stratification and Lexicase selection In our experiments, simple Lexicase selection [?] is used to select parents for crossover and mutation. Size controls are implemented by stratifying the population in size buckets, and lexicase selection is done inside the bucket. This not only reduces the complexity of lexicase selection by reducing the number of candidates, but makes the comparisons size-fair, i.e., parents are only competing against parents of the same size.

Size strata are chosen at random: the first stratum uniformly, and in the case of crossover, the second parent uniformly within plus or minus 10 complexity points from the first parent.

The resulting offspring is placed in the stratum corresponding to their structural complexity. After this, to keep the strata equally occupied, the stratum with most individuals is selected. A naturally elitist inverse tournament of size 2 on hits is held to identify a member for deletion.

Initialization To create an initial population, probabilistic tree creation [?] is used. If population size allows, every stratum is initialized with the same number of initial trees. There is no attempt to find optimal constants of these trees. They are initialized as regular trees with random constants drawn from a Cauchy distribution by dividing two independent draws from a Gaussian (`randn()/randn()`).

Variation Insert locations are chosen using Koza’s 90/10 rule, and subtree crossovers are applied 80% of the time; otherwise a size fair subtree mutation is used.

With this setup, the only difference between a regular symbolic regression system and the system under study is the aligned-insert routine.

6.1 Estimating computational effort

Comparing different symbolic regression systems for their computational efficiency to reach a certain level of performance is not straightforward. Standard metrics such as the number of individuals processed ignore important factors such as the number of fitness cases, the number of evaluations per individual, and the complexity of the individuals. A run that keeps average population size low can run many orders faster than a run that is allowed to bloat. Counting individuals processed does not distinguish those

Therefore, many comparisons are done on wall-clock time [?, ?], but this is hardware, language, and implementation dependent. Although it is suitable for comparing alternative approaches within the same symbolic regression engine, programmed by the same developer, it does not allow for algorithmic comparison.

A natural measure for computational effort is the number of evaluations performed. It would therefore be tempting to use the size of the population times the number of fitness cases as a measure of effort. Every new individual processed would be considered to be freshly evaluated. However, evaluations can be cached, and dramatic speedups can be achieved doing this, particularly when caching is used with algorithms that heavily share subtrees (e.g., use subtree crossover almost exclusively) [?].

A useful observation is that under assumptions of optimal caching, only new subtrees need to be evaluated. It is furthermore useful to ignore duplication effects where the same subtree is inserted at the same location in the same tree, and assume that every subtree inserted in a tree will lead to new evaluations from the insertion point upwards to the root.

This gives a clear set of measurements to track.

A subtree crossover at depth d will add dn to the overall evaluation count, and a subtree mutation inserting an (assumed new and unique) subtree of size s at depth d will add $(d+s)n$ to the overall evaluation count. These mutations are thus more expensive than crossovers. Computational effort is then measured in node-evaluations, or **nods** for short, and effort will be measured in mega-nods (Mnods).

Under this definition of computational effort, an aligned insert, optimizing constants, costs $dn \log n$, with subtree mutation adding an additional sn , thus costing $sn + dn \log n$. Finding optimal constants as studied here, costs a factor of $\log n$, and for the method to be considered succesful, it will need to outperform standard methods in a comparable computational budget.